

MENDOv3: AN IMPROVED AI-ENABLED MEDICINE RECOMMENDER FOR AN OVER THE COUNTER (OTC) DRUG DISPENSER

Kristine Mae Adlaon¹, Jayrald Basan², Earl Cedric M. Coronado³, Geoffrey J. Diapana³, and John Paul J. Estano³

1. Faculty Researcher, College of Computer Studies
2. Faculty Researcher, College of Pharmacy and Chemistry
3. Student Researcher, College of Computer Studies

I. INTRODUCTION

Self-medication using over-the-counter (OTC) drugs is a widespread practice across many parts of the world, particularly in communities where healthcare access is limited or where individuals seek faster and more affordable treatment for minor illnesses [12, 13]. Studies have shown that people frequently rely on their interpretation of symptoms when selecting medications, which can lead to inappropriate drug use when symptoms are misunderstood or inaccurately described. In the Philippines, self-medication is also commonly practiced due to factors such as limited healthcare accessibility, long waiting times in medical facilities, and the affordability and availability of OTC medicines in local pharmacies [14, 15, 16]. This trend is reflected in national findings showing that only 7.4% of Filipinos report not self-medicating, meaning roughly 92.6% engage in some form of self-treatment [15]. In Davao City, despite the Botika ng Bayan program providing free medicines to over a million residents annually, individuals often continue to rely on prior experience, advice from family members, or information obtained online when deciding which medication to take [17]. These behaviors highlight the need for accessible tools that can assist users in interpreting symptoms and identifying appropriate OTC medications.

To support this need, previous research introduced Mendo v1.0 and Mendo v2.0, systems designed to detect symptoms from user-described inputs and recommend suitable OTC medicines. These systems utilized a retrieval-based architecture grounded in information retrieval techniques, particularly TF-IDF, to match user symptom descriptions with predefined symptom categories and medication recommendations. The approach demonstrated promising performance in retrieving relevant symptom matches from user inputs.

Despite the need for tools that assist with symptom interpretation, one of the key challenges in supporting self-medication lies in how individuals describe their symptoms. Symptoms are subjective experiences expressed through natural language, which can vary widely across individuals and contexts. Studies have shown that ambiguity in natural language makes automated detection difficult, as patients often combine multiple symptoms or use negations in their descriptions [18, 19]. In the Philippines, AI-based symptom checker systems have reported similar challenges, where informal phrasing and mixed-language expressions reduce system accuracy [19, 20]. Locally, in Davao City, patients frequently describe

symptoms using Bisaya–English expressions, further complicating automated interpretation and highlighting the need for localized NLP models tailored to regional language use.

However, the earlier version of the system (v1.0) was significantly limited by its single-token symptom processing, English-only interface, low storage capacity, and bulky hardware configuration. Version 2.0 addressed several of these constraints through multilingual dictionary-based translation, improved dispensing capacity, and the use of TF-IDF with cosine similarity for text matching. Despite these improvements, evaluation revealed a critical precision–safety gap: the system achieved high recall (97.63%) but considerably lower precision (78.08%), indicating that roughly one out of five recommendations could be clinically inappropriate. Further analysis showed that the dataset used to train the recommendation engine was limited and manually augmented, while the multilingual component relied heavily on synonym-based dictionary lookups. In addition, the system treated symptoms as isolated keywords rather than clinically distinct concepts. For example, cough symptoms may be classified as dry (non-productive) or wet (productive), which require different therapeutic approaches despite sharing the same base symptom. These limitations highlight a broader semantic and clinical context gap in the v2.0 architecture, ultimately reducing the reliability and safety of its medication recommendations.

To examine the limitations of the baseline system, an adversarial challenge set of 110 scenarios was constructed. The scenarios were designed to target common failure conditions in symptom interpretation and medication recommendation, including negation handling, contraindication awareness, multi symptom interpretation, and pediatric constraints. These cases reflect situations in which simple keyword retrieval methods often struggle to produce appropriate recommendations.

Table 1 presents the performance of the baseline system across these categories. The baseline model, which relies on TF IDF vectorization and cosine similarity for symptom matching, demonstrates moderate performance on straightforward retrieval tasks but performs poorly when the input requires compositional interpretation or safety awareness. Accuracy declines notably in multi symptom and pediatric scenarios, and the system frequently fails to prevent contraindicated medication suggestions when relevant conditions are present.

Category	Total	Correct	Failed	Accuracy
Baseline	20	9	11	45.0%
Contraindication	22	8	14	36.4%
Multi-symptom	16	5	11	31.2%

Negation	11	6	5	54.5%
Pediatric	7	2	5	28.6%

Table 1: Performance by category

These quantitative results are further illustrated by several qualitative failures. In one example, the system recommended a fever tablet for the input “I only have sore throat, NO fever,” rather than suggesting throat specific treatments such as lozenges or sprays. In another case, the system returned an adult tablet brand for a four year old child with cough and fever who was allergic to aspirin, instead of recommending pediatric syrup formulations. Similarly, a Bisaya input that explicitly negated phlegm still resulted in a recommendation for a product intended for productive cough. Representative examples of these failures are summarized in Table 2.

Category	Language	Input Text	Expected	Predicted
Negation	English	I only have sore throat, NO fever	lozenges or throat spray	Biogesic
Pediatric	English	Child 4 yrs with cough and fever, allergic to aspirin	acetaminophen_syrup; guaifenesin_syrup or dextromethorphan_syrup	Solmux
Multi-Symptom Decomposition	Tagalog	May ubo na may plema at may lagnat	Guaifenesin; acetaminophen	Solmux
Contraindication	English	Toothache, has high blood pressure	Acetaminophen; avoid decongestants	Neozep

Table 2: Selected failure examples

These findings underscore a well-documented limitation of retrieval-based architectures: systems that lean on keyword matching cannot reason about what a patient is actually saying, cannot enforce safety constraints like age restrictions or contraindications, and

struggle with anything beyond single-symptom, single-language queries [21, 22, 23]. In multilingual environments such as the Philippines, these challenges are further amplified by informal language use and mixed-language inputs [24, 25, 26]. Therefore, there is a need for improved architectures that can better capture the semantic relationships between symptoms, incorporate safety-aware reasoning, and provide more reliable medication guidance in real-world scenarios.

Findings from the systematic literature review indicate that recent studies have proposed several architectural strategies to address the limitations of retrieval-based symptom recommendation systems. While traditional TF-IDF-based approaches provide strong lexical matching for symptom terms, researchers have noted that such systems often struggle with semantic interpretation, complex symptom descriptions, and safety-aware reasoning in clinical recommendation tasks.

Several studies have explored enhanced symptom understanding techniques by integrating contextual semantic representations with rule-based linguistic processing. For example, Tan et al. (2024) and Kalva and Padakanti (2024) demonstrate that contextual embeddings can improve semantic similarity detection in patient narratives, while rule-based negation and scope handling remain useful for accurately interpreting patient-generated inputs, particularly in multilingual settings [2, 27]. This hybrid approach allows systems to capture deeper semantic relationships while maintaining interpretability and computational efficiency.

Other researchers have proposed multi-symptom decomposition and ranking strategies to address the limitations of single-label symptom matching. Studies by Sohail et al. (2025) and Bodele et al. (2024) suggest decomposing complex symptom descriptions into independent clinical components before generating recommendations [28, 29]. Rather than forcing a single best match, these approaches enable systems to produce complementary recommendations that correspond to multiple co-occurring symptoms, improving the clinical relevance of treatment suggestions.

Another important direction in the literature involves incorporating structured medical knowledge into recommendation pipelines. Systems such as RECOMED, described by Zomorodi et al. (2024), integrate knowledge graphs that encode relationships between active ingredients, contraindications, and patient-specific constraints such as age or pre-existing conditions [30]. These structured representations allow systems to enforce deterministic safety checks, including the prevention of contraindicated drug combinations and age-inappropriate formulations.

Collectively, these approaches highlight a growing shift toward hybrid architectures that combine lexical retrieval, semantic interpretation, and structured medical knowledge to improve both accuracy and safety in clinical recommendation systems.

It is also worth noting that commercial symptom checker applications like Ada Health, Babylon Health, and Isabel Healthcare have shown strong diagnostic accuracy in controlled evaluations, often using proprietary datasets with millions of clinical cases and large-scale

model training. However, these systems are designed for broad diagnostic triage rather than OTC medication recommendation, they require constant internet connectivity, they operate mostly in English, and they rely on proprietary cloud-based architectures that raise data privacy concerns under Philippine law (Republic Act No. 10173). Mendo v3.0 is different from these systems in several important ways: it targets a narrowly scoped OTC recommendation task (13 symptoms, 24 medicines), runs entirely offline on constrained hardware, supports Filipino multilingual input including Bisaya and Tagalog, and prioritizes deterministic safety guarantees over broad diagnostic coverage.

Building on the approaches identified in the systematic literature review and addressing the limitations observed in the previous Mendo versions, this study introduces MendoVendo v3.0, an improved OTC medication recommendation system designed to enhance symptom interpretation and safety-aware decision support. While earlier versions relied primarily on TF-IDF vectorization and cosine similarity for symptom matching, the proposed system adopts a hybrid architecture that integrates lexical retrieval with complementary mechanisms for semantic interpretation, multi-symptom handling, and safety validation.

Specifically, the proposed architecture incorporates three key components. First, an enhanced symptom understanding module integrates contextual semantic representations with rule-based negation and scope detection to improve the interpretation of user-described symptoms, particularly in mixed-language inputs commonly observed in the Philippine context. To further improve the clinical interpretation of symptoms and enhance recommendation safety, the system incorporates elements of the OLDCARTS symptom assessment framework — specifically the Character, Onset, Aggravating, Timing, and Duration components — to guide targeted clarification questions for ambiguous symptoms and enforce time-based safety limits [31, 32]. This structured representation allows the system to capture additional contextual information about symptoms that may influence appropriate medication selection, while also screening for cases that have persisted long enough to warrant professional medical evaluation rather than OTC self-treatment.

Second, a multi-label symptom decomposition and ranking pipeline separates complex symptom descriptions into independent symptom clusters, enabling the system to generate complementary medication recommendations when multiple conditions are present. Third, a clinical constraint engine encodes structured relationships between OTC medications — including active ingredients, contraindications, and age-based formulation rules — through a curated JSON dataset, enabling deterministic safety checks during the recommendation process.

Objectives of the Study

This study aims to develop MendoVendo v3.0, an AI-enabled multilingual symptom detection and over-the-counter (OTC) medicine recommendation system designed to support safe and accessible self-medication. Specifically, the study aims to:

1. Design and implement a multilingual natural language processing framework capable of detecting and interpreting user-described symptoms, including linguistic

challenges such as negation, abbreviations, multi-symptom expressions, and mixed-language inputs, while incorporating structured symptom assessment through the OLDCARTS framework, including a duration-based safety check that screens for symptoms persisting beyond clinically safe thresholds for OTC self-medication.

2. Develop a recommendation mechanism that suggests appropriate OTC medicines based on identified symptoms while incorporating safety-aware filtering mechanisms to reduce inappropriate medication recommendations.
3. Release the annotated dataset as a publicly accessible CSV file hosted on a recognized academic data repository (such as Zenodo or Kaggle Datasets), along with documentation describing the annotation schema, symptom label definitions, and intended use cases to support reproducibility.
4. Develop an accessible automated platform that enables users to receive safe OTC medicine recommendations based on their reported symptoms.

II. METHODS

This chapter explains the methods used to design, build, and evaluate Mendo v3.0, a multilingual symptom detection and over-the-counter medicine recommendation system intended for pharmacy-kiosk use. The software was developed in Python using Flask and was designed for offline or low-connectivity operation on modest hardware. The central goal of the study was to improve the safety and precision of symptom-based OTC recommendations while preserving support for the kinds of multilingual, code-switched, and informal inputs commonly used by Filipino pharmacy customers.

Mendo v3.0 was developed as a clear departure from the earlier Mendo approach, which relied on direct text-to-drug matching. In the present version, the task is separated into two more transparent steps: symptom identification first, then recommendation scoring based on those symptoms. This design makes it easier to handle multi-symptom input, negated statements, clarification prompts, and safety checks in a way that can be traced and explained.

The study is organized as a three-iteration design process. Iteration 1 corresponds to the completed baseline system currently implemented in the codebase, including the OLDCARTS-inspired clarification logic, safety filtering, consultation logging, and internal benchmark evaluation. Iteration 2 is planned as a controlled real-user validation phase in selected pharmacy settings, where users will interact with the Iteration 1 system and their consultation inputs will be saved as logs for expert review. Iteration 3 is planned for expert-guided refinement of the system using the annotated consultation corpus, followed by re-evaluation and final deployment of the improved version. Because this chapter documents both the completed baseline artifact and the intended later research phases, the sections below distinguish clearly between implemented Iteration 1 components and proposed Iteration 2 and Iteration 3 activities.

Research Design

The study follows an iterative design science research approach. This was a natural fit because the project does not simply test an existing technique; it involves defining, building, evaluating, and refining a working artifact through multiple cycles until it meets functional and safety standards. The outcome of the research is therefore both a working system and a documented set of design decisions that justify the architecture [\[1,3\]](#).

The research design is structured into three iterations.

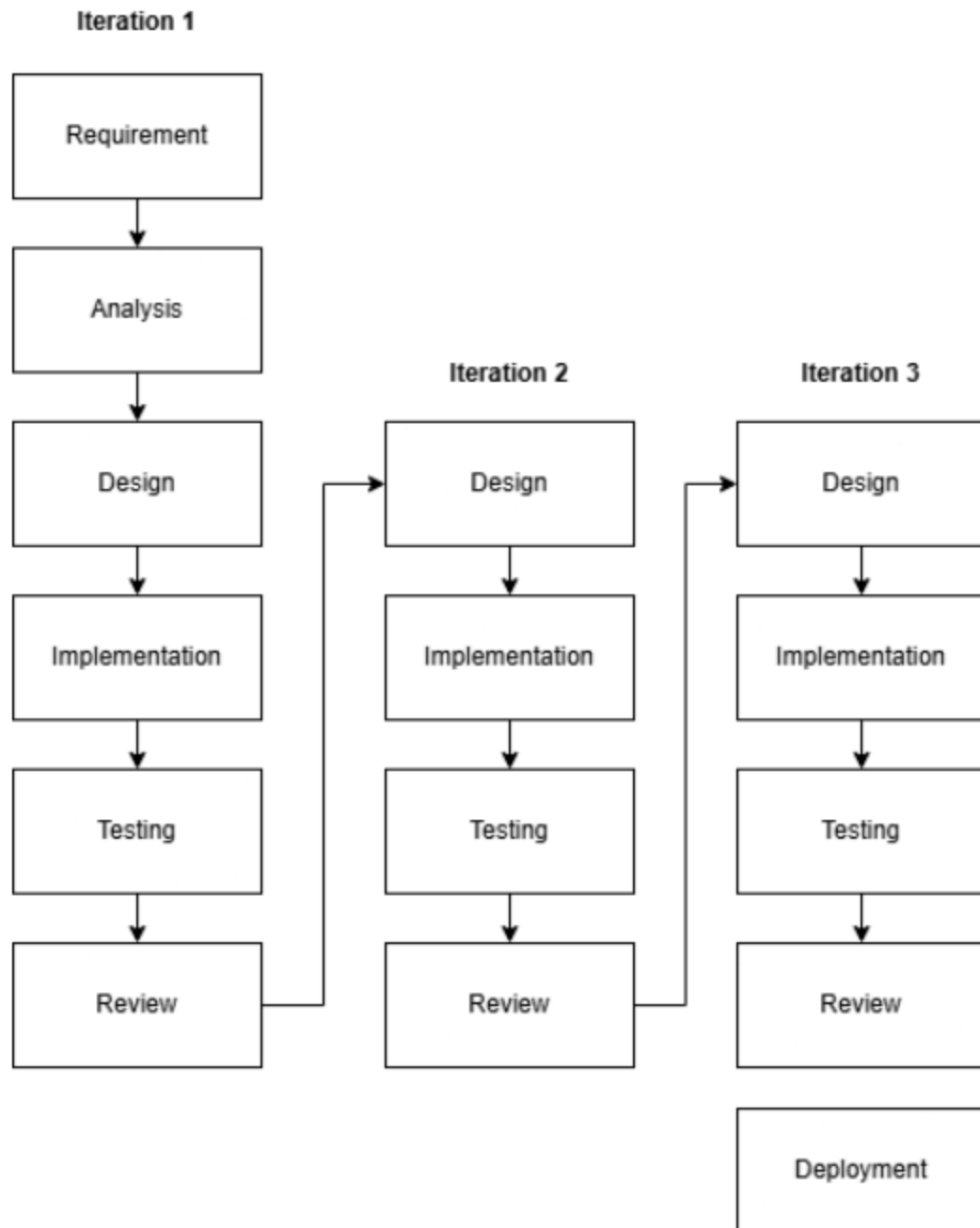


Figure 1: Iterative Research Approach

Iteration 1: System Design, Development, and Internal Evaluation

The first iteration covers the completed baseline system currently implemented in the repository. In this phase, the focus was on turning the earlier direct text-to-drug approach into a safer and more transparent symptom-first recommendation system that could already

handle clarification, logging, and internal evaluation.

Requirement. The baseline system must accept natural-language symptom descriptions, identify OTC-related complaints within the system's scope, apply red-flag screening for emergencies, ask clarification questions when symptom details are unclear, and return medicine recommendations from a structured OTC dataset. Since the target deployment is a pharmacy kiosk in Davao City, the system must also support multilingual and code-switched inputs (Bisaya, Tagalog, English, and combinations thereof) and log every consultation for later review. In parallel, the researchers aim to collect approximately 5,000 symptom-response entries from individuals purchasing OTC medicines at selected Botika ng Bayan pharmacies. These entries should cover the study's predefined base symptoms: fever, cough, cold, headache, allergy, stomachache, and body pain.

Analysis. Based on these requirements, the researchers designed a five-stage hybrid pipeline: (1) a red-flag triage layer, (2) dictionary-based symptom extraction, (3) a semantic embedding fallback for unseen inputs, (4) a hybrid filtering stage for negation and safety checks, and (5) a recommendation stage connected to a structured medicine dataset. The design also includes consultation logging so that every interaction (detected symptoms, safety flags, recommendation outcomes) is stored for later validation. For data collection, a purpose-built interface was designed to present structured symptom-related questions while allowing respondents to describe symptoms in their own words.

Design. Based on the requirements outlined above, the design phase focused on translating those functional needs into concrete architectural decisions. The researchers chose a five-stage pipeline architecture that separates symptom detection from recommendation scoring, keeping each concern modular and testable. For multilingual support, the design combines a curated phrase dictionary (covering Tagalog, Bisaya, English, Taglish, and Jejemon) with a semantic embedding fallback for inputs the dictionary cannot handle. Safety requirements were addressed through a dedicated triage layer at the front of the pipeline, a centralized safety filter at the end, and a duration-based screening step that checks how long each detected symptom has persisted before proceeding with OTC recommendations. The clarification mechanism was modeled after the OLDCARTS clinical assessment framework, adapted to only ask follow-up questions for symptom attributes that are genuinely ambiguous (such as cough type or diarrhea context) and to screen for symptoms that have lasted beyond the safe limits for self-medication.

Implementation. During implementation, the researchers developed the baseline system as a Flask-based consultation application. This phase included the construction of a multilingual symptom dictionary, preparation of the semantic anchor set, integration of triage and negation logic, and organization of the structured OTC medicine dataset for recommendation use. An OLDCARTS-inspired clarification layer was also implemented, covering context-sensitive complaints such as cough, diarrhea, headache, stomachache, and runny nose. In addition, a duration safeguard was built into the recommendation flow as a universal safety measure applied across all nine symptom categories. For each detected symptom, the system asks the user how long the complaint has persisted, using bilingual multiple-choice buttons. If the reported duration exceeds a clinically defined threshold — for instance, more than three days for fever or more than fourteen days for cough — the system blocks all OTC recommendations for that symptom and advises the user to consult a licensed medical professional instead. Through this process, the system became a functional working prototype rather than just a conceptual model. For the data collection side, the researchers plan to deploy the data collection interface during visits to selected

Botika ng Bayan pharmacies in Davao City once ethical clearance has been secured. Participants who agree to take part in the study will answer symptom-related questions through the system, and their responses will be stored in a database until the target of approximately 5,000 entries is reached within the planned four-week collection period. This target was set based on comparable multilingual symptom dataset studies in low-resource languages, which typically range from 2,000 to 10,000 entries for initial corpus development [18, 25], and was adjusted to account for the expected daily foot traffic at the selected pharmacy branches over the collection period.

Testing. After implementation, the system underwent structured testing and validation. The baseline system was evaluated through structured benchmark cases, regression checks, and scenario-based safety validation. These tests verified whether the system could correctly recognize symptoms, suppress unsafe recommendations when red flags were present, and trigger clarification prompts when additional context was needed. Testing also confirmed that the modules work effectively as a complete pipeline rather than as isolated components. On the dataset side, the collected responses will undergo data cleaning and filtering once collection is completed. Entries that do not meet the inclusion criteria, such as those containing unrelated symptoms, incomplete information, or irrelevant text, will be removed. The cleaned dataset will then serve as the initial foundation for the system's symptom recognition and recommendation processes.

Review. At the end of the first iteration, the researchers reviewed the system output to check whether it met the expected standards. The system evaluation confirmed 288/288 exact matches on the structured benchmark, 9/9 exact matches on the semantic stress set, and 72/80 (90%) exact matches on the real-user simulation set, with zero outright failures across all 377 test cases. Once the data collection phase is completed, the dataset will also be assessed to confirm that it meets the required size and quality criteria and that it adequately covers the predefined base symptoms. This review will help determine whether the first iteration has established a solid enough baseline system and dataset to be refined and expanded in the following iterations.

Iteration 2: Expert Annotation, Hardware Development, and Real-User Validation

Iteration 2 has two major tracks running concurrently. The first track focuses on expert annotation of the datasets collected in Iteration 1 and real-user validation of the software system. The second track involves the design and development of the hardware vending (vendo) dispenser that will eventually house the software system for autonomous OTC medicine dispensing.

Because this phase involves real participants in an actual pharmacy setting, ethical clearance is built into the iteration itself. Before any live consultation is recorded, the study must secure approval from the appropriate institutional ethics review body and obtain permission from participating Botika ng Bayan branches. Field activities begin only after the approved protocol, participant information materials, and data-handling procedures are in place [10, 11].

Design. The software track is designed around controlled field deployment of the Iteration 1 system in selected pharmacy settings. Real users interact with the consultation interface while the system records their inputs, clarification responses, detected symptoms,

red-flag triggers, and recommendation outputs. These consultation logs constitute the primary real-world dataset for this iteration. Participation is voluntary, and each participant is informed about the study's purpose, what data is recorded, how logs are stored, and how they will be used. Consent materials are prepared in languages relevant to the study setting. The hardware track involves designing the vendo machine—a physical kiosk that integrates the Mendo software system, a touchscreen interface, a medicine dispensing mechanism, and inventory management. The hardware design must support the system's offline operation requirements and be suitable for placement in pharmacy settings.

Implementation. On the software side, the completed Iteration 1 system is deployed under supervised conditions. Participants who consent enter their symptom descriptions through the system, and logs are stored as the primary evaluation dataset. Privacy measures are observed: consultations are conducted to minimize bystander exposure, and access to logs is limited to authorized research team members and the designated expert reviewer. On the hardware side, the researchers begin fabricating the vendo prototype—designing the dispensing mechanism, integrating the Raspberry Pi 5 computing unit, wiring the touchscreen interface, and assembling the physical enclosure. The hardware must pass basic functional tests (reliable dispensing, correct inventory tracking, power management) before integration with the software system.

Testing. The consultation logs are reviewed by a licensed pharmacist who annotates the correct symptoms, appropriate safety actions, and expected medicine recommendations for each logged interaction. This expert-annotated corpus allows direct comparison between the system's actual outputs and expert judgment, revealing which case types the baseline handles well and which still need improvement. Concurrently, the hardware prototype undergoes mechanical and electrical testing to ensure reliable medicine dispensing and inventory accuracy.

Review. The main outputs of Iteration 2 are (1) an expert-annotated consultation corpus from real user interactions and (2) a functional hardware vendo prototype. The review assesses whether the software system remained reliable under authentic pharmacy language, whether the annotated data is sufficient for system refinement, and whether the hardware prototype meets the physical requirements for deployment.

Iteration 3: System Refinement, Hardware Integration, and Final Deployment

Iteration 3 brings together the improved software and the completed hardware into a single integrated system.

Design. This iteration draws on findings from the expert-annotated logs and hardware testing results from Iteration 2. The researchers identify which symptom expressions were frequently missed, which clarification rules need strengthening, and which recommendation cases require tighter safety handling. If the annotated dataset proves large and reliable enough, the semantic symptom-matching component may be adapted using the Iteration 2 consultation corpus, though the recommendation core remains grounded in a structured medicine dataset rather than a black-box model.

Implementation. Software refinements may include expanding the multilingual dictionary, adjusting semantic thresholds, improving clarification prompts, refining duration thresholds based on pharmacist feedback and observed referral patterns, strengthening rule-based safety checks, and updating the medicine dataset based on expert feedback. On

the hardware side, the refined software is integrated into the vendo machine. This includes connecting the recommendation output to the dispensing mechanism, so that once a user accepts a recommendation, the system triggers the correct medicine slot. Inventory management, payment handling, and user interface refinements are finalized during this phase.

Testing. The integrated system undergoes a comprehensive evaluation: the existing structured benchmark and regression tests are re-run, the system is evaluated against held-out annotated consultation data from Iteration 2, and end-to-end dispensing tests verify that the correct medicine is physically dispensed for each recommendation. The aim is to confirm that both software accuracy and hardware reliability meet deployment standards.

Review. The final review assesses whether symptom recognition has improved, whether safety behavior remains stable, whether the hardware performs reliably, and whether the integrated system is mature enough for operational use.

Deployment. If the refined system meets the required performance, safety, and usability standards, final deployment proceeds at the end of Iteration 3. In this study, deployment means releasing the validated MendoVendo system—software and hardware together—for its intended pharmacy-kiosk setting.

Research Philosophy

This study is grounded in a pragmatic research philosophy. The project sits at the intersection of language processing, software engineering, and clinical safety, which means the value of the artifact cannot be judged by algorithmic accuracy alone—it must also behave safely and sensibly when real people use it in a real pharmacy [1,3].

The pragmatic stance is reflected throughout the evaluation strategy. Quantitative testing measures symptom detection performance on structured benchmarks, while design decisions are guided by whether they improve traceability, multilingual robustness, and safe recommendation behavior. The system is judged not only by whether it classifies text correctly, but by whether its outputs are appropriate for an OTC self-medication context.

Theoretical Framework

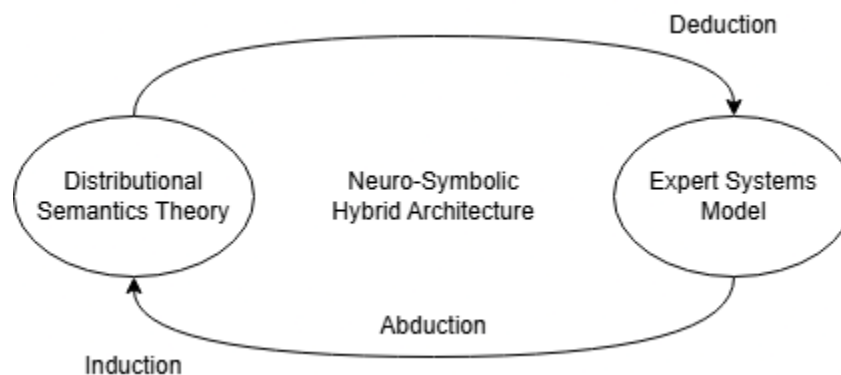


Figure 2: Theoretical Framework

Mendo v3.0 is built on a hybrid architectural approach that deliberately combines connectionist AI with symbolic reasoning. This design addresses the precision–safety gap observed in previous retrieval-based iterations by keeping probabilistic linguistic interpretation separate from deterministic clinical decision-making. The system's five-stage pipeline rests on two complementary theoretical foundations: Distributional Semantics Theory and the Knowledge-Based Expert Systems model [1, 5, 6]. These operate in a cascaded arrangement, the first ensures the system can understand what users are saying despite linguistic diversity, while the second ensures the system's medical reasoning stays within safe, verifiable bounds.

Distributional Semantics Theory

The system's ability to make sense of unstructured, multilingual, and informally phrased symptom descriptions draws from Distributional Semantics Theory. In plain terms, this theory holds that the meaning of a word emerges from the company it keeps — words appearing in similar contexts tend to carry similar meanings [5, 7]. This matters because pharmacy-kiosk users in Davao City do not type clinical terminology; they describe symptoms using paraphrases, regional dialects, or code-switched language that no finite dictionary can fully anticipate.

Within the Mendo v3.0 architecture, this theory underwrites the semantic embedding fallback layer. Driven by inductive learning from large textual corpora, a pre-trained multilingual transformer model maps user input into a high-dimensional vector space. When evaluating live user input, this layer employs abductive reasoning to infer the most probable underlying concept from informal text, thereby enabling the system to identify meaning equivalence between different surface expressions. Consequently, phrases such as "sakit ng ulo" and "my head is pounding" both map near the HEADACHE concept. This maintains broad recall for diverse patient language without requiring exact dictionary matches. Critically, however, the system treats these probabilistic outputs as supporting evidence rather than as authoritative detections. Every semantic suggestion must still pass through lexical guards that prevent the model from hallucinating symptoms not actually present in the user's text [7, 8].

Knowledge-Based Expert Systems Theory

While distributional semantics handles the variability of human language, the system's core safety and reasoning mechanisms are grounded in the Expert Systems model. An expert system mimics the decision-making of a human specialist by applying structured knowledge through a deterministic inference engine [1, 6, 9]. By executing strict deductive reasoning, the Expert Systems model governs both the symbolic processing of natural language, including negation rules, scope detection, and contrastive boundary logic, as well as the clinical reasoning required for safe medication dispensing, such as contraindication checks, age-based formulation selection, and paracetamol overlap detection.

This deterministic foundation is necessary for a safety-critical, domain-specific application. Rather than operating as an open-ended conversational agent, the system functions as a strictly constrained decision-support tool, mapping validated symptoms to a predefined over-the-counter catalog using absolute clinical rules [1, 6].

Conceptual Framework

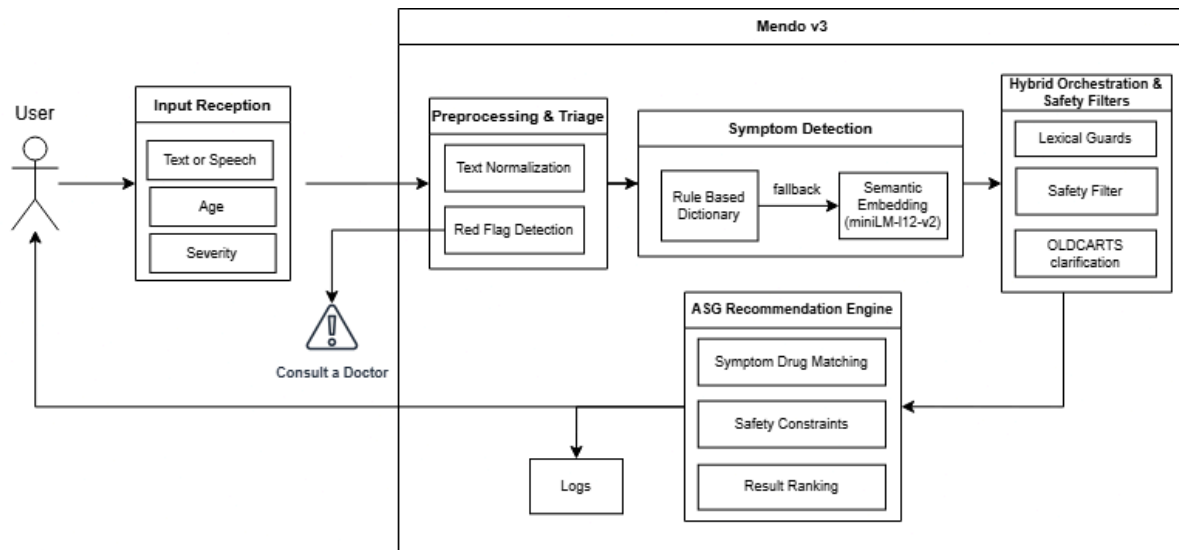


Figure 3: Conceptual Framework

Mendo v3.0 is designed as a multilingual OTC medicine recommendation kiosk that accepts free-text symptom descriptions and produces age-appropriate, safety-screened medication suggestions. The system follows a five-stage conceptual process.

In the first stage, the user enters their symptom description through the kiosk interface using Bisaya, Tagalog, English, or any combination. Age and symptom severity can be optionally provided.

In the second stage, the system preprocesses and normalizes the input while simultaneously screening for red-flag conditions—potential emergencies that warrant professional medical attention rather than OTC treatment. If a serious condition is flagged, the system suppresses all OTC recommendations and advises the user to seek immediate medical help. Only inputs that pass this screening proceed further.

In the third stage, the system performs symptom detection using a hybrid NLP approach that combines rule-based dictionary matching with semantic analysis. This enables recognition of both direct symptom mentions and varied, informal, or multilingual expressions of the same complaint.

In the fourth stage, detected symptoms pass through additional validation: negation checks, context-sensitive screening, and clarification prompts for symptoms that require more information (such as whether a cough is dry or productive). The output is a validated symptom set.

In the fifth stage, validated symptoms first pass through a duration screening step where the user is asked, for each detected symptom, how long the complaint has persisted. If the reported duration exceeds a clinically defined threshold specific to that symptom, the system blocks OTC recommendations and refers the user to a healthcare professional. Only symptoms that fall within safe self-medication timeframes proceed to the recommendation engine, which matches them against the OTC medicine database using symptom relevance, age restrictions, contraindications, and safety rules. The system presents appropriate recommendations with dosage guidance and safety reminders—or, if the case exceeds the safe scope of OTC use, advises the user to consult a healthcare professional.

The key conceptual shift from earlier Mendo versions is this: rather than predicting a medicine directly from raw text, v3.0 first identifies what the user's symptoms are, then evaluates possible medicines under defined safety constraints. This two-step approach provides a clearer and safer foundation for handling multi-symptom cases, negation, multilingual input, and context-sensitive recommendations.

System Architecture

Mendo v3.0 is implemented as a Flask application running on Python 3.12.3. The system comprises a customer-facing consultation layer, a hybrid NLP and recommendation layer, a data and knowledge layer, an inventory integration layer, and a logging layer. The consultation interface accepts typed symptom descriptions and can also process transcribed speech through an optional server-side endpoint. Results are filtered by age and cross-referenced against available inventory before recommendations are shown.

The architecture deliberately favors deterministic, rule-based processing for the first pass of symptom detection and safety screening. Semantic embeddings serve as a fallback only when dictionary matching finds nothing. This arrangement prioritizes traceability and safety in a pharmacy context where the system must explain and constrain its outputs rather than generate open-ended medical advice.

Stage 0: Triage / Red-Flag Safety Layer

The initial processing stage functions as a critical safety mechanism, screening user input for medical emergencies that require professional medical intervention rather than OTC self-medication. To balance precision and recall, the triage layer avoids brittle exact-phrase matching and overly broad keyword matching. Instead, it utilizes Proximity-Based Co-occurrence and Exclusion Rules.

The system detects critical red-flag conditions by searching for specific combinations of clinical entities occurring within a defined lexical window. Crucially, this stage incorporates "Exclusion Tokens" (negative constraints) to prevent over-triage. This ensures that severe keywords (e.g., "blood" or "numb") do not trigger false alarms when used in non-emergency, treatable contexts (e.g., "high blood pressure" or "numb tooth"), allowing those cases to safely proceed to the OTC recommendation stage.

Category	Clinical Rationale	Trigger Pattern (Co-occurrence)	Lexical Exceptions (Ignore if present)
Gastrointestinal Hemorrhage	Lower/Upper GI bleed	[blood / dugo] + [stool / dumi / vomit / suka]	None
Cardiac Emergency	Possible myocardial infarction	[pain / masakit] + [chest / dibdib / dughan]	halak, plema, ubo, cough (Allows treatment for chest muscle pain caused by severe coughing/congestion)
Respiratory Emergency	Impaired airway / Anaphylaxis	[difficulty / hirap / lisod] + [breathing / hinga]	None
Suspected Dengue / Viral	High risk of hemorrhage	[fever / lagnat] + [rashes / pantal / spots]	kagat, bite, allergy (Allows OTC antihistamines for insect bites/mild allergies)
Neurological Event (Stroke)	Requires immediate ER evaluation	[numb / manhid] + [face / mukha / half / kalahati]	ngipin, tooth (Allows OTC analgesics for radiating dental pain)
Severe Dehydration	Acute kidney injury risk	[diarrhea / nagtatae] + [walang ihi / no urine]	None
Pregnancy Contraindication	High risk of fetal harm from OTCs	[buntis / pregnant / naglilihi] (Single Keyword)	asawa, misis, kapatid, sister (Allows proxy purchasing for a pregnant relative)
Dangerous Hyperthermia	High risk of convulsions	[fever / lagnat / temp] + [40, 41, 42]	kilo, kg, edad, years old (Prevents numbers from being misclassified as temperature)

Direct Emergencies	Immediate intervention required	[seizure / kombulsyon] OR [nahimatay / fainted]	None
--------------------	---------------------------------	---	------

Table 3: Common Red Flag Key Words with Clinical Rationale

Stage 1: Dictionary-Based Symptom Extraction

The primary extraction stage uses a curated multilingual symptom dictionary containing 331 phrases across 13 symptom labels, covering five language variants. Each phrase was manually verified for linguistic accuracy and clinical relevance through iterative expansion during Iteration 1.

Symptom Label	Phrase Count	Languages	Key Examples
HEADACHE	42	Tag/Bis/Eng	"masakit ang ulo", "labad akong ulo", "headache"
FEVER	43	Tag/Bis/Eng	"lagnat", "nilalagnat", "hilanat", "init akong lawas"
BODY_ACHES	39	Tag/Bis/Eng	"masakit katawan", "binugbog", "bug at akong lawas"
RASHES	32	Tag/Bis/Eng	"pantal", "rash", "makati ang balat"
STOMACH_ACHE	38	Tag/Bis/Eng	"sakit tiyan", "kabag", "hyperacidity"
SORE_THROAT	31	Tag/Bis/Eng	"masakit lalamunan", "sore throat", "garas"

COUGH_PRODUCTIVE	20	Tag/Bis/Eng	"may plema", "basang ubo", "halak"
ALLERGIC_RHINITIS	17	Tag/Bis/Eng	"allergy", "bahing", "alerdyi", "makati ilong"
COUGH_GENERAL	16	Tag/Bis/Eng	"ubo", "cough", "inuubo", "gi-ubo"
COUGH_DRY	15	Tag/Bis/Eng	"walang plema", "tuyong ubo", "dry cough"
DIARRHEA	17	Tag/Bis/Eng	"pagtatae", "diarrhea", "lbm", "loose bowel"
NASAL_CONGESTION	11	Tag/Bis/Eng	"barado ilong", "stuffy nose", "blocked nose"
RUNNY_NOSE	10	Tag/Bis/Eng	"sipon", "runny nose", "tumululo ilong"

Table 4: Symptom Dictionary — 331 phrases across 13 labels

Text Normalization. All input text undergoes deterministic normalization before phrase matching: (1) lowercase conversion, (2) leetspeak/Jejemon character substitution (@ → a, 0 → o, 1 → i, 3 → e, 4 → a, 5 → s, 7 → t, 8 → b, \$ → s, ! → i, | → i), (3) removal of non-alphanumeric characters (preserving ñ for Filipino words), and (4) whitespace collapsing. This handles informal digital communication styles common among Filipino youth (e.g., "s@k1t ul0" normalizes to "sakit ulo" and matches HEADACHE).

Phrase Matching Strategy. Multi-word phrases use substring matching in the normalized text; single-word keywords use word-boundary regular expressions (\b anchors) to prevent false positives from partial string matches (e.g., preventing "tubo" from matching the COUGH keyword "ubo").

Negation Handling. Patients frequently mention symptoms they do not have alongside those they do (e.g., "masakit ang ulo ko pero walang lagnat"—"my head hurts but I don't have a fever"). The system addresses this through: a negation lexicon of 11 words covering Tagalog, Bisaya/Cebuano, and English; a proximity window requiring the negation word

within 0–2 words before the symptom phrase; consumed negation logic where an intervening symptom keyword absorbs the negation before it reaches a farther symptom; and a negated-label tracking set that prevents the fuzzy rescue mechanism from re-adding symptoms that were explicitly negated.

Contrastive Boundary Splitting. Input is split on contrastive conjunctions (pero, but, kaso, however, though) into independent segments. Each segment is evaluated independently for negation, with positive mentions after a contrastive boundary overriding prior negation.

Cough-Type Qualification. A seven-step decision tree distinguishes COUGH_DRY, COUGH_PRODUCTIVE, and COUGH_GENERAL before the general dictionary scan runs. The tree incorporates itchy/scratchy throat indicators, chest-rattle/congestion indicators, cough-word verification, cough negation with plema-filler exemption, dry qualifier detection, wet qualifier detection, and a COUGH_GENERAL default.

Fuzzy Rescue Mechanism. After dictionary matching and negation processing, a fuzzy rescue pass attempts to recover misspelled symptom mentions using Levenshtein distance matching across seven rescue families.

Target Symptom	Fuzzy Targets	Max Distance	Exclusion Set
FEVER	lagnat, fever, hilanat	1	None
RUNNY_NOSE	sinisipon, sinasipon, sisipon, sipon	2	None
COUGH	ubo	1	ulo, ulu, ole, olo, tubo, ubos, ubi, ube, tuba, ubod
DIARRHEA	pagtatae, nagtatae, kalibang	2	kaninang, kanina, kaninag
RASHES	pantal, butlig	1	ipantal, pantalon, pantalan
STOMACH_ACHE	tiyan, sikmura	1	Requires nearby pain word

BODY_ACHES	katawan, lawas	1	Requires nearby pain word
------------	----------------	---	---------------------------

Table 5: Summary of Levenshtein Distance Matching

Proximity-Based Heuristics. Several symptom labels use co-occurrence detection for compositional descriptions: headache heuristics (head word + pain word within ≤ 5 tokens), sore throat proximity (throat word + pain word within ≤ 5 tokens with dual negation checking), per-cue-group nasal inference, and an allergen-trigger heuristic.

Stage 2: Semantic Embedding Fallback

The semantic fallback covers symptom descriptions that fall outside the dictionary's 331 curated phrases. It uses the pre-trained paraphrase-multilingual-MiniLM-L12-v2 model from the Sentence-Transformers library (Reimers & Gurevych, 2019), a compact transformer (~33 million parameters) producing 384-dimensional sentence embeddings.

The model is used in inference-only mode without any fine-tuning. This is a deliberate choice motivated by three factors: (1) safety — fine-tuning on a small corpus risks overfitting while degrading performance on unseen linguistic variants; (2) reproducibility — the pre-trained model is a fixed, publicly available artifact; and (3) multilingual coverage — the model's pre-trained knowledge spans 50+ languages including Filipino and Cebuano, which would be difficult to replicate with limited domain-specific fine-tuning.

At initialization, 118 anchor sentences (5–12 per symptom label) are pre-encoded into 384-dimensional vectors. At runtime, the user's input is encoded and compared against all anchors using cosine similarity. Symptoms with a best-anchor score ≥ 0.65 are returned as candidates. The 0.65 threshold was determined empirically through iterative testing.

To give more context on how we landed on 0.65: we started by testing a low threshold of 0.40, which produced way too many false positives since a lot of unrelated inputs were clearing that bar. Bumping it up to 0.50 helped, but still let through phrases that were not really symptom-related. At 0.60, most valid paraphrases were being captured with fewer false positives, but we noticed that some legitimate matches for figurative or idiomatic expressions were getting dropped. At 0.65, the system kept coverage for meaningful paraphrases while filtering out adversarial cases like gibberish input and non-symptom text. Going higher to 0.70 and above caused the system to reject valid multilingual paraphrases that fell just barely below the cutoff. So 0.65 ended up being the best middle ground between catching real symptoms and not making things up. We do plan to run a more formal sensitivity analysis during Iteration 3 once we have real consultation data to work with.

A key design decision is the precision-first switching logic: the semantic layer activates only when Stage 1 finds zero symptoms. When the dictionary returns results, those results stand as-is, with no semantic augmentation. Deterministic, auditable results take priority over probabilistic detections wherever dictionary coverage is sufficient.

One trade-off of this design is that if the dictionary catches the primary symptom but misses a secondary one that was expressed idiomatically or figuratively, the semantic layer will not activate for that case. We accepted this trade-off because in an OTC recommendation context, a false positive (detecting a symptom that is not there) is more dangerous than a false negative (missing a secondary symptom). A missed secondary symptom leads to an incomplete but not wrong recommendation, whereas a false positive could trigger an unnecessary or even contraindicated medicine. The 8 partial matches in the 80-case real-user simulation are a direct reflection of this design choice — in all 8 cases, the primary symptom was correctly detected, but a secondary one was missed. Relaxing this logic so the semantic layer runs as an augmentation rather than a fallback is something we plan to explore in Iteration 3, once we have enough real-user data to evaluate whether it increases false positive rates.

The model loads as a lazy singleton instantiated only on the first query requiring semantic processing to minimize startup time and memory on the Raspberry Pi 5 target hardware.

Stage 3: Hybrid Merge, Lexical Guards, and Safety Filters

This stage orchestrates results from Stages 0–2 and applies multi-layered filtering before anything reaches the recommendation engine. The processing flow:

1. Run triage detection (Stage 0) → red_flags list
2. Run dictionary extraction (Stage 1) → dict_symptoms
3. Apply explicit cough negation override
4. If dict_symptoms is non-empty → return dict_symptoms (skip semantic)
5. If dict_symptoms is empty → load semantic model (lazy singleton)
6. Run semantic extraction → raw semantic detections
7. Apply lexical guard: 11 keyword-gate groups covering all 13 symptom labels — each semantic detection must be grounded in at least one relevant keyword from the original input text
8. Apply centralized safety filter: seven explicit negation functions plus red-flag suppression rules
9. Score-rank and select top-N (default: 2) semantic symptoms
10. Return final symptom set with source attribution

Lexical Guard Mechanism. This prevents the embedding model from "hallucinating" symptoms not lexically present in the input. Each of 13 symptom labels has a dedicated keyword gate. For example, a semantic detection of COUGH requires at least one

cough-related keyword in the input. COUGH_PRODUCTIVE has a compound gate requiring both a cough keyword AND a phlegm keyword. Detections failing their keyword gate are silently dropped.

Centralized Safety Filter. Seven explicit negation functions (fever, headache, cough, diarrhea, sore throat, nasal symptoms, allergy) are applied using word-boundary-safe regex. Two red-flag suppression rules prevent dangerous false positives: blood_in_stool → suppress DIARRHEA; severe_allergic_reaction → suppress SORE_THROAT.

OLDCARTS Clarification Layer and Duration Screening Layer. The system uses targeted clarification logic to collect additional context for symptoms where the label alone is insufficient for a safe recommendation—such as cough type disambiguation, diarrhea context, or runny nose triggers. The clarification layer is treated as a practical assessment heuristic within the recommendation design, not as a core theoretical pillar [31, 32]. Beyond these context-specific clarifications, the system also implements the Duration component of the OLDCARTS framework as a universal safety measure applied across all nine symptom categories. After any applicable clarification questions are resolved, the system asks the user how long each detected symptom has persisted, presenting bilingual multiple-choice options (for example, "1–2 days / 1–2 araw", "3 days / 3 araw", "More than 3 days / Higit sa 3 araw" for fever). If the reported duration exceeds the clinically defined threshold for that particular symptom, the system blocks all OTC recommendations and displays a referral notice advising the user to consult a licensed healthcare professional, along with the clinical basis for the referral.

The duration thresholds were derived from clinical guidelines published by the World Health Organization [33, 36, 38, 42], the United States Centers for Disease Control and Prevention [34, 39], the Mayo Clinic clinical reference database [35, 37, 40, 41], and standard OTC package insert warnings used in Philippine pharmacies. The 14-day cough threshold aligns with the DOH community-level TB screening protocol, given that the Philippines ranks among the top three countries in global TB burden [38]. The 3-day fever threshold accounts for the dengue-endemic status of the country, where dengue warning signs typically emerge within 24–48 hours after fever resolution [33, 34]. The 2-day diarrhea threshold follows established clinical advice that adult diarrhea lasting beyond 48 hours warrants medical evaluation due to dehydration risk and the possibility of bacterial or parasitic infection [35, 36].

Symptom	Threshold	Action if Exceeded	Clinical Rationale
Fever	> 3 days	Block OTC, refer	Dengue, typhoid, malaria indicator (endemic PH) [33, 34]
Diarrhea	> 2 days	Block OTC, refer	Dehydration risk; bacterial or parasitic

			infection [35, 36]
Sore Throat	> 5 days	Block OTC, refer	Streptococcal infection; rheumatic fever risk [37]
Stomach Ache	> 7 days	Block OTC, refer	Peptic ulcer, gallstones, appendicitis
Headache	> 7 days	Block OTC, refer	Hypertension, neurological issues, medication overuse headache [41, 42]
Body Aches	> 7 days	Block OTC, refer	Inflammatory arthritis, nerve damage, post-viral sequelae
Rashes	> 7 days	Block OTC, refer	Fungal infection, scabies, chronic immune condition
Allergic Rhinitis	> 7 days	Block OTC, refer	Chronic immune issue requiring prescription treatment
Nasal Congestion	> 10 days	Block OTC, refer	Bacterial sinusitis [40]
Runny Nose	> 10 days	Block OTC, refer	Bacterial sinus infection [40]
Cough (all types)	> 14 days	Block OTC, refer	TB screening protocol (DOH, endemic PH) [38, 39]

Table 6: Duration Safeguard Matrix — Symptom-Specific Thresholds for OTC Self-Medication

The duration check is performed per symptom. When the system detects multiple symptoms, it asks about each one sequentially, displaying a progress indicator (for example, "Symptom 1 of 3") so the user knows how many questions remain. If even one symptom exceeds its threshold, that symptom's recommendations are blocked and a referral is shown, while symptoms within safe timeframes proceed to the recommendation engine. This design ensures that cases where OTC treatment is still appropriate for some symptoms are not unnecessarily blocked entirely.

OLDCARTS Sample Implementation. To further illustrate the pipeline's multilingual capability and multi-step OLDCARTS flow, consider a second scenario. A Cebuano-speaking user approaches the kiosk and types: "gi ubo ko" (I'm coughing).

The hybrid extraction pipeline processes this Bisaya input through three stages. The dictionary scanner matches "ubo" against the Cebuano cough entries, producing the symptom label COUGH_GENERAL. Because cough is the sole detected symptom and no contextual clues about cough type (dry or productive) are present in the input, the system triggers a Character clarification question:

"Please spec/ify: Is your cough dry (walay/walang plema) or with phlegm (naay/may plema)?"

The user selects "Dry cough." This maps to the OLDCARTS Character component — identifying the quality of the symptom. The system now reclassifies the symptom as COUGH_DRY, which changes the recommendation strategy: it selects cough suppressants such as Tuseran Forte (Dextromethorphan + Paracetamol + Phenylephrine + Chlorphenamine) and Sinecod Forte (Butamirate citrate), while excluding expectorants like Solmux (Carbocisteine), because administering an expectorant for a dry, non-productive cough serves no therapeutic purpose and can cause unnecessary side effects.

Before displaying these recommendations, the system applies the Duration component — the Timing element of OLDCARTS. The user is presented with four bilingual multiple-choice buttons tailored to the cough threshold:

"Gaano na katagal ang iyong tuyong ubo (dry cough)? / How long have you had this dry cough?"

- Less than a day
- 1–7 days
- 8–14 days
- More than 2 weeks

If the user selects "More than 2 weeks," the system blocks the OTC recommendation and displays a referral notice: cough persisting beyond 14 days may indicate chronic bronchitis, undiagnosed asthma, or tuberculosis, conditions requiring professional medical evaluation. If the user selects any of the other three options (less than a day, 1–7 days, or 8–14 days), the duration falls within the safe threshold and the system proceeds to display the medicine recommendations normally.

Had the user instead selected "With phlegm" during the Character clarification, the system would have classified the symptom as COUGH_PRODUCTIVE and recommended expectorants — Ascof Forte (Lagundi leaf extract), Solmux (Carbocisteine), and Robitussin (Guaifenesin) — while excluding suppressants. The same 14-day duration check would still apply.

This cough scenario demonstrates three key system capabilities working in sequence: (1) multilingual symptom extraction from Cebuano input, (2) OLDCARTS Character clarification to distinguish clinically different cough subtypes, and (3) OLDCARTS Timing enforcement through the Duration Safeguard to prevent OTC self-medication for chronic symptoms.

Stage 4: ASG Recommendation Engine

The final stage maps detected symptoms to OTC medicine recommendations using a structured JSON dataset built under the Authoritative-Source-Grounded (ASG) framework.

ASG Framework and Medicine Dataset. The ASG framework was adopted after the originally planned pharmacist annotation process could not be completed within the project timeline. Under this framework, all medicine data is sourced exclusively from authoritative pharmaceutical references: package inserts, MIMS Philippines, and DOH Philippines guidelines. The present version therefore represents a constrained, authoritative-source-grounded baseline; pharmacist review and expert annotation remain part of the Iteration 2 validation cycle, and final integrated validation is planned for Iteration 3. Each of the 24 medicine entries contains 15 fields:

Field	Type	Source
Brand	String	Product label
Generic/Main Use	String	Package insert

Drug Category	String	MIMS classification
Primary Symptom	String	Clinical mapping
Typical Symptoms Treated	String (comma-sep)	Package insert
Dosage Form	String	Product label
Minimum Age	String	Package insert
Notes	String	Clinical notes
Approved_Indications	Tuple[str]	Package Insert / MIMS
Indication_Source	String	ASG attribution
Contraindications	Tuple[str]	Package Insert / MIMS
Warnings	Tuple[str]	Package Insert
Drug_Interactions	Tuple[str]	Package Insert / MIMS
Max_Duration_Days	Integer	Package Insert
Contraindication_Source	String	ASG attribution

Table 7: Summary of ASG Framework and Medicine Datasets

The dataset contains 24 medicine entries covering 19 unique brands, representing common OTC medications available in Philippine pharmacies. Multiple entries exist for medicines available in different dosage forms (e.g., Biogesic Tablet and Biogesic Syrup) to enable age-appropriate form selection.

Recommendation Logic. The algorithm follows a deterministic, rule-based process:

1. Triage Gate: If red flags were detected, suppress all OTC recommendations and return "CONSULT A LICENSED MEDICAL EXPERT" with the specific emergency category.
2. COUGH_GENERAL Handling: If COUGH_GENERAL is the sole detected symptom, request cough-type clarification rather than recommending a generic medicine.
3. Symptom-to-Medicine Matching: Each detected symptom is matched against the dataset using rule-based scoring, and candidates are ranked by match score.
4. Brand Merging: Multiple entries of the same brand are merged, keeping the highest score and combining match reasons.
5. Safety Checks: (a) Paracetamol overlap detection warns when multiple paracetamol-containing medicines are recommended; (b) opposing mechanism warnings flag simultaneous expectorant and suppressant recommendations; (c) age filtering removes medicines below the user's minimum age requirement; (d) duration screening blocks OTC recommendations when the user reports symptom persistence beyond the clinically defined safe threshold for that symptom category (see Table 6: Duration Safeguard Matrix).

Medicine Coverage. The 24 entries cover: Cold & Flu (Bioflu), Cold/Decongestant (Neozep, Decolgen, Decolgen Forte, Symdex-D), Cough Suppressant (Tuseran Forte, Sinecod Forte), Expectorant (Ascof Forte, Solmux, Robitussin), Pain & Fever (Biogesic, Advil), Allergy (Cetirizine), Anti-Diarrheal (Loperamide/Diatabs), Probiotic (Erceflora), Rehydration (Hydrite ORS), Antacid (Kremil-S), and Antispasmodic (Buscopan).

Multilingual Language Support

Mendo v3.0 processes patient symptom descriptions in five language variants reflecting the linguistic diversity of Philippine pharmacy interactions.

Language Variant	Coverage Level	Example Input
Tagalog	Primary	"masakit ang ulo ko at nilalagnat"
Bisaya/Cebuano	Primary	"labad akong ulo, garas akong tilaok"
English	Full	"I have a headache and fever"
Taglish/Conyo (code-switch)	Full	"masakit head ko, may cough ako"
Jejemon/Leetspeak	Full	"s@k1t ul0", "Ignat ako"

Table 8: Coverage of the Multilingual Language Support for Mendo v3.0

Tagalog and Bisaya/Cebuano receive primary support with extensive phrase coverage in the dictionary, dedicated negation words, and anchor sentences in the semantic layer. English is fully supported through both dictionary phrases and the embedding model's native capability. Taglish/Conyo code-switching is handled through combined dictionary coverage and cross-lingual embedding similarity. Jejemon/leetspeak is handled through normalization-layer character substitution rules.

Evaluation Methodology

Benchmark Design. The evaluation framework uses a structured CSV-based benchmark containing 288 test cases organized across 60 categories and four tiers of increasing difficulty:

Tier 1: Core Functional (100 tests) — Single-symptom detection, multiple symptoms, basic negation, partial negation, noisy input, misspelling, alternative phrasing, English input, code-switching, third-person descriptions, temporal markers, age context, severity descriptors, question forms.

Tier 2: Extended Robustness (100 tests) — Jejemon/leetspeak, Bisaya-heavy descriptions, compound multi-symptom cases, contrastive negation, realistic run-on input, diagnostic confusion, interjections, symptom chains, temporal progression, formal register, extreme severity, complex English, heavy code-switching, pharmacy kiosk realistic.

Tier 3: Adversarial (64 tests) — Universal negation, false positive traps, sore throat edge cases, contrastive boundary variants, multi-negation, mixed complex scenarios, non-symptom input, single word input, stress tests, Bisaya negation, reversed word order, Conyo/slang, gibberish detection, post-negation constructs, long realistic descriptions, allergy-nasal disambiguation.

Tier 4: Triage Safety (24 tests) — Chest pain, breathing difficulty, blood symptoms, loss of consciousness, seizure, severe allergic reaction, high fever thresholds, mixed emergency/symptom cases.

Tier 5: Duration Safeguard (31 tests) — Duration input parsing for various time expressions, question generation coverage across all symptom categories, safe-pass verification for durations within threshold, blocked-referral verification for durations exceeding threshold, exact-boundary edge cases at each threshold limit, API response format validation for safe, blocked, and multi-symptom chaining scenarios, and threshold completeness checks confirming all 13 symptom–duration entries are present.

Evaluation Metrics

Exact Match. Detected symptoms exactly equal expected symptoms (set equality). This is the primary success metric.

Partial Match. At least one correct symptom detected with no false positives, but incomplete capture.

Failed. Wrong symptoms detected or empty result when non-empty output was expected.

F1 Score. Harmonic mean of per-test precision and recall, averaged across all cases.

Additional Validation Sets

9-Case Semantic Stress Set: Figurative, metaphorical, and culturally idiomatic descriptions testing the semantic fallback's handling of non-literal language.

80-Case Real-User Simulation: Realistic patient-style inputs mimicking actual pharmacy interactions, including rambling descriptions, emotional language, and multiple complaints in a single utterance.

Evaluation Results

Benchmark	Exact Match	Partial	Failed
288-case structured	288/288 (100%)	0	0
9-case semantic stress	9/9 (100%)	0	0
80-case real-user sim	72/80 (90%)	8	0
Combined (377 total)	369/377 (97.9%)	8	0

Table 9: Summary of Evaluation Results for Each Match

Table 8 and Table 9 show different numbers on the same 288 test cases because they evaluate different stages of the pipeline. In Table 8, when a test case involves cough, the benchmark simulates the user answering the cough clarification question ("Is your cough dry or with phlegm?"), resolving COUGH_GENERAL into the specific cough type. Under that condition, all 288 cases achieve exact match. In Table 9 (the comparative benchmark), this clarification step is removed so that all six models are evaluated on the same raw pipeline output. Without the clarification, 54 cough-related cases are scored as partial mismatches because the system detects COUGH_GENERAL instead of the specific type, yielding 234/288 (81.2%) exact match and a micro F1 of 0.852. The raw-output evaluation in Table 9 is the fairer comparison because the baseline models do not have a clarification mechanism.

In the 80-case real-user simulation, 8 cases came out as partial matches — the system caught the main symptom but missed a secondary one that was buried in vague or heavily misspelled phrasing. Across all three evaluation sets (377 total cases), zero cases resulted in outright failure, meaning the system never detected a wrong symptom.

The 10-point difference between the structured benchmark (100% exact match) and the real-user simulation (90%) is worth discussing. The structured benchmark, while it covers a wide range of linguistic and adversarial categories, was put together by the same researchers who built the system's rules and dictionary. Even though no benchmark case was copied from the phrase inventory, our familiarity with the system's capabilities might introduce some implicit bias in how the test cases were designed. The real-user simulation was specifically built to mimic how people actually talk at a pharmacy kiosk — with heavy

misspelling, texting shorthand, run-on sentences, and emotional language — and that is where the system's limitations start to show.

Looking at the 8 partial matches more closely, two patterns keep coming up. First, severely misspelled body-part words that fall outside what the fuzzy rescue mechanism can handle (for example, "bdy" for "body" or "tultunlan" for "lalamunan"). Second, inputs where the user mentions both a productive cough and a general cough in the same sentence, which the system's cough decision tree cannot handle because it only assigns one cough type per input. These are specific things we can address — the first through dictionary expansion, the second through modifying the cough logic to support dual cough-type detection — and both are on the list for Iteration 3.

Overall, we think the real-user simulation results should be treated as the more realistic indicator of how the system would actually perform once deployed.

Comparative Benchmarking Against Transformer and ML Baselines

A recurring question during development was whether the hybrid pipeline was unnecessarily complex. A straightforward fine-tuned transformer, the kind of model that dominates NLP leaderboards, might simply outperform the hand-built pipeline given the same test cases. To answer this, the system was benchmarked against five baseline models spanning both transformer and traditional ML approaches.

Models Compared. Six models were evaluated on the 288-case benchmark:

1. **Mendo Hybrid Pipeline.** The complete five-stage system (dictionary extraction + MiniLM semantic fallback + safety filters), using the paraphrase-multilingual-MiniLM-L12-v2 model (~33M parameters) in inference-only mode.
2. **XLM-RoBERTa (fine-tuned).** xlm-roberta-base (~278M parameters), fine-tuned for multi-label symptom classification with a linear classification head over the [CLS] token.
3. **mBERT (fine-tuned).** bert-base-multilingual-cased (~178M parameters), fine-tuned with the same multi-label classification setup.
4. **SVM (TF-IDF).** Linear Support Vector Classification with character-level n-gram TF-IDF features (ngram range 2–5, max 10,000 features), wrapped in a OneVsRest multi-label strategy.
5. **Logistic Regression (TF-IDF).** L-BFGS logistic regression with the same TF-IDF feature extraction and OneVsRest wrapping.
6. **Random Forest (TF-IDF).** 200-tree ensemble classifier with the same TF-IDF feature configuration.

Evaluation Protocol. The Mendo pipeline was evaluated directly on the full set of 288

benchmark cases because it does not learn from the benchmark corpus during evaluation. Its symptom detection relies on pre-defined handcrafted rules and a pre-trained semantic embedding model used only for inference, rather than supervised fitting on the test set. In contrast, the five baseline models required training and were therefore evaluated using stratified 5-fold cross-validation, yielding approximately 230 training cases and 58 testing cases per fold. For the transformer baselines, fine-tuning was performed for 10 epochs per fold using the AdamW optimizer, a learning rate of 5×10^{-5} , weight decay of 0.01, batch size of 16, and maximum sequence length of 128. For the TF-IDF baselines, character-level n-grams from 2 to 5 were used with a vocabulary cap of 10,000 features.

Fairness and Transparency. Several measures were taken to ensure a fair comparison. All six models were evaluated against the same 288 test cases. The five baselines shared identical cross-validation splits (random seed = 42), the same TF-IDF extraction settings, and the same training hyperparameters within each model class. Notably, the baselines had a structural advantage the Mendo pipeline did not: they trained on roughly 80% of the benchmark per fold (~230 samples), whereas the Mendo pipeline was evaluated cold. Its 331-pharse symptom dictionary was built from clinical references, pharmacy fieldwork, and multilingual language resources, not from the benchmark CSV. The comparison therefore tests an architectural question: does hand-encoded domain knowledge outperform supervised learning from limited labeled data?

Model	Exact Match	Micro Precision	Micro Recall	Micro F1	Hamming Loss	Latency (ms)
Mendo Hybrid Pipeline	234/288 (81.2%)	0.856	0.849	0.852	0.028	93.07
SVM (TF-IDF)	161/288 (55.9%)	0.860	0.636	0.731	0.045	0.05
Random Forest (TF-IDF)	144/288 (50.0%)	0.837	0.605	0.702	0.049	23.77
mBERT (fine-tuned)	110/288 (38.2%)	0.839	0.409	0.550	0.064	35.45
Logistic Regression (TF-IDF)	95/288 (33.0%)	0.902	0.154	0.263	0.082	0.06

XLM-RoBE RTa (fine-tuned)	65/288 (22.6%)	1.000	0.017	0.033	0.094	24.14
---------------------------------	-------------------	-------	-------	-------	-------	-------

Table 10: Comparative Benchmark Results

Analysis. The results showed a clear gap under the constraints of this study. The hybrid pipeline achieved 81.2% exact match and a micro F1 of 0.852, outperforming the next-best model (SVM) by over 25 percentage points. The transformer models performed well below what one might expect given their size. XLM-RoBERTa, despite having 278 million parameters, managed only 22.6% exact match. mBERT did somewhat better at 38.2%, but that is still less than half of what the hybrid pipeline achieved.

That said, these results come with an important caveat: the comparison is fundamentally limited by data scarcity, and we want to be upfront about that. Transformer models are built to learn from huge amounts of labeled data — typically thousands or tens of thousands of examples. In our setup, each fold only gave the transformers roughly 230 training samples spread across 13 symptom labels, which creates a combinatorial space of $2^{13} = 8,192$ possible label combinations. That is nowhere near enough for a transformer to learn meaningful patterns. XLM-RoBERTa shows this clearly: it achieved 1.000 precision but only 0.017 recall, which means it basically learned to predict almost nothing rather than risk getting something wrong. While that kept its Hamming loss low, a model that only catches 6 out of 357 actual symptom labels is not usable in any real sense. So the low transformer scores here reflect a data insufficiency problem, not some inherent flaw in transformer architectures. Given enough training data and proper GPU hardware, the results would very likely look different.

SVM with character n-gram TF-IDF turned out to be the strongest traditional baseline, reaching 55.9% exact match and an F1 of 0.731. Character-level n-grams work well for Filipino text because they capture subword patterns and can tolerate minor misspellings. But SVM hits a hard ceiling: it cannot detect negation, handle code-switching between Bisaya and English mid-sentence, or reason about multiple symptoms mentioned together. These are exactly the things that account for the hybrid pipeline's 25-point lead.

Taken together, these results show that given Mendo's specific constraints — a small dataset of 288 labeled cases across 13 symptom labels, multilingual code-switched input, safety-critical negation handling, and offline deployment on a Raspberry Pi 5 with 8 GB RAM and no GPU — the hand-engineered hybrid approach was the most suitable architecture among those we tested. This benchmark was not designed to prove that rule-based systems are generally better than transformers. It answers a practical engineering question: given the resources we actually have access to, does hand-encoding domain knowledge work better than learning from a small labeled corpus? For this project, the answer is yes. Down the line, if larger annotated datasets become available through the Iteration 2 data collection, revisiting transformer-based approaches with techniques like data augmentation, few-shot prompting, or transfer learning from related medical NLP datasets would be a worthwhile next step.

Benchmark Integrity. No benchmark case was copied from or derived from the handcrafted

phrase inventory, and the 288-case benchmark set was finalized only after the rule inventory and semantic anchor set had been frozen. The benchmark includes adversarial categories (negation, code-switching, misspellings, gibberish) specifically designed to test failure modes the dictionary cannot cover, further reducing the risk of indirect overfitting between the rule base and the evaluation set.

Why Not a Large Language Model for Drug Recommendation?

After seeing how well transformers perform in other medical NLP tasks, a reviewer might reasonably ask: why not go all the way and use a full generative LLM like GPT-4, LLaMA, or Gemini to handle both symptom detection and drug recommendation? We considered this early in the project and ultimately decided against it, for reasons that go beyond benchmark scores.

1. **Safety and determinism.** When a system recommends medicine, a wrong answer can cause real harm. LLMs produce text probabilistically. Run the same prompt twice and you might get different advice. They can hallucinate drug names, miss a negation in the input, or suggest a medicine that is contraindicated for a child. Our pipeline avoids this by design: paracetamol overlap checks, opposing-mechanism warnings, age-based formulation filters, red-flag triage suppression, and duration-based referral thresholds are all hard-coded rules that fire on every single consultation without exception. An LLM does not natively provide those deterministic guarantees unless you wrap it in an external rule layer on top, and at that point you are essentially rebuilding what we already have [\[1, 6, 21, 22\]](#).
2. **Explainability and traceability.** If a pharmacist or a panel reviewer asks "why did the system recommend this drug?", we need a clear answer. Our pipeline logs exactly which dictionary phrases matched, which semantic anchors activated, which negation rules blocked a detection, and which safety filters altered the final output. Every recommendation has a paper trail. An LLM generates a text response, but there is no straightforward way to trace back which internal weights or attention patterns produced that specific recommendation. You could bolt on an explanation layer, but that adds complexity without matching the native traceability the rule-based pipeline already provides [\[3, 6\]](#).
3. **Offline deployment on constrained hardware.** Our target device is a Raspberry Pi 5 with 8 GB of RAM and no GPU, sitting in a pharmacy that may have spotty internet at best. That rules out cloud LLM APIs right away. You cannot have the kiosk go down every time the Wi-Fi drops. Running an LLM locally is not much better: even heavily quantized models like Phi-3 Mini (3.8B parameters) or LLaMA 3.2 (1B parameters) demand more memory and compute than the Pi can offer. Our MiniLM semantic model, at 33 million parameters, fits comfortably in under 500 MB and returns results in about 93 milliseconds per query on CPU. The gap between 33 million and even 1 billion parameters is simply too large for the hardware we are working with.
4. **Cost and sustainability.** Cloud LLM APIs charge per token, and a busy pharmacy kiosk could easily handle hundreds of consultations a day. Paying an API provider to recommend Biogesic or cetirizine, which cost only a few pesos each, does not make economic sense. Once deployed, the Mendo pipeline has zero per-query cost. No API keys, no monthly bills, no external service dependencies.

5. **Regulatory and data privacy.** Sending patient symptom descriptions to an external API (whether Google's, OpenAI's, or anyone else's) raises immediate privacy concerns under Philippine data privacy law (Republic Act No. 10173). Health information routed through third-party servers creates compliance risks that we would rather not take. A fully offline system sidesteps the issue entirely: patient data never leaves the kiosk [10, 11].
6. **The problem is bounded, not open-ended.** LLMs shine when the task requires flexible reasoning across broad knowledge domains. That is not what Mendo does. We detect symptoms from 13 fixed labels, look them up against 24 medicines, and apply age and safety rules. It is a narrow, well-defined problem, and narrow problems benefit more from purpose-built engineering than from general-purpose generation. The benchmark bears this out: transformers with 178 to 278 million parameters still could not match a pipeline specifically designed for this scope, which tells us that throwing more parameters at the problem is not the answer when the domain is small and the rules are clear.

For all these reasons, we chose a hybrid architecture: a compact 33-million-parameter embedding model handles the linguistic understanding, while hard-coded rules handle every safety-critical decision. It is a deliberate division of labor—we trust the model to understand language, but we do not trust any model to decide what medicine someone should take.

Error-Driven Iterative Refinement

During Iteration 1, the system was tested by deliberately feeding it tricky and unusual inputs to find weaknesses. This process revealed 19 issues spread across common problem areas like the system misunderstanding negation (e.g., "wala nako'y ubo" being read as a cough instead of "no cough"), mixing up similar symptom types, incorrectly identifying symptoms from unrelated words, and missing certain safety-critical patterns. Each issue was fixed one at a time, and after every fix the full 288-case benchmark was re-run to make sure nothing that was already working got broken.

As a supplementary probe—not a substitute for structured testing—the researchers also used Gemini 2.5 Pro to scan for potential blind spots the team might have overlooked. Out of seven areas it flagged, six had already been addressed through the testing process. This supplementary check gave additional confidence that the iterative approach had caught most of the important edge cases, though the benchmark and regression tests remain the primary basis for evaluating system correctness.

Project Workplan

[illegible]

4.2	Hardware–Software Integration											
4.3	Integrated System Testing and Evaluation											
4.4	Final System Validation											
4.5	Deployment of the MendoVendo System											
5	Documentation & Thesis Defense											
5.1	Thesis Writing & Documentation											
5.2	Revision & Adviser Consultation											
5.3	Thesis Presentation Consultation											
5.4	Final Thesis Defense											

Legend:

	Phase 1: Protocol Development & Ethics Approval		Phase 2: Iteration 1
	Phase 3: Iteration 2		Phase 4: Iteration 3
	Phase 5: Documentation & Defense		

Proposed Budget

Item No.	Item Description	Quantity	Unit Cost (₱)	Total Cost (₱)
A. Equipment/Hardware				
1	Pre-owned commercial vending unit (small, capsule/snack type)	1	5,000–12,000	5,000–12,000
2	Raspberry Pi 5 (8GB)	1	5,000	5,000
3	7" Touchscreen	1	3,500	3,500
4	MicroSD + Power Supply	1	1,200	1,200
5	Retrofit wiring (GPIO → existing motor)	1 lot	1,000–2,000	1,000–2,000
6	Relay module to bridge RPi ↔ vendo motors	1	500–800	500–800
B. Supplies and Materials				
7	Printing of consent forms and data collection sheets	5 reams	300.00	1,500.00

Item No.	Item Description	Quantity	Unit Cost (₱)	Total Cost (₱)
8	Printer inks	3 set	1,200.00	3,600.00
9	Folders, Envelopes, and Labels	1 lot	500.00	500.00
10	Stationery (pens, clipboards, markers)	1 lot	500.00	500.00
C. Transportation and Fieldwork				
11	Transportation	3 months	1,500	4,500.00
12	Meals/Sustenance during fieldwork	3 months	800.00	2,400.00
13	Contingency fund (unforeseen expenses)	1 lot	2,000.00	2,000.00
TOTAL ESTIMATED BUDGET:				₱35,000

References:

- [1] Eguia, H., Sánchez-Bocanegra, C. L., Vinciarelli, F., Alvarez-Lopez, F., & Saigí-Rubió, F. (2024). Clinical decision support and natural language processing in medicine: Systematic literature review. *Journal of Medical Internet Research*, 26, Article e55315. <https://doi.org/10.2196/55315>
- [2] Tan, J., Rong, Y., Zhao, K., Bian, T., Xu, T., Huang, J., Cheng, H., & Meng, H. (2024). Natural language-assisted multi-modal medication recommendation. *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, 2200–2209. <https://doi.org/10.1145/3627673.3679529>
- [3] Bayor, A. A., Li, J., Yang, I. A., & Varnfield, M. (2025). Designing clinical decision support systems (CDSS)—A user-centered lens of the design characteristics, challenges, and implications: Systematic review. *Journal of Medical Internet Research*, 27, e63733. <https://doi.org/10.2196/63733>
- [4] Wohlgemut, J. M., Erhan Pisirir, Stoner, R. S., Perkins, Z. B., Marsh, W., Tai, N. R. M., & Kyrimi, E. (2024). A scoping review, novel taxonomy and catalogue of implementation frameworks for clinical decision support systems. *BMC Medical Informatics and Decision Making*, 24(1). <https://doi.org/10.1186/s12911-024-02739-1>
- [5] Elvas, L. B., Almeida, A., & Ferreira, J. C. (2025). Natural language processing in medical text processing: A scoping literature review. *International Journal of Medical Informatics*, 204, 106049. <https://doi.org/10.1016/j.ijmedinf.2025.106049>
- [6] Newton, N., Bamgboje-Ayodele, A., Forsyth, R., Tariq, A., & Baysari, M. T. (2025). A systematic review of clinicians' acceptance and use of clinical decision support systems over time. *npj Digital Medicine*, 8(1). <https://doi.org/10.1038/s41746-025-01662-7>
- [7] Haemmerl, K., Fastowski, A., Libovický, J., & Fraser, A. (2023). Exploring anisotropy and outliers in multilingual language models for cross-lingual semantic sentence similarity. *Findings of the Association for Computational Linguistics: ACL 2023*, 7023–7037. <https://doi.org/10.18653/v1/2023.findings-acl.439>
- [8] Mahajan, Y., Bansal, N., Blanco, E., & Karmaker, S. (2024). ALIGN-SIM: A task-free test bed for evaluating and interpreting sentence embeddings through semantic similarity alignment. *Findings of the Association for Computational Linguistics: EMNLP 2024*, 7393–7428. <https://doi.org/10.18653/v1/2024.findings-emnlp.436>
- [9] Lin, X., Liang, C., Liu, J., Lyu, T., Ghumman, N., & Campbell, B. (2024). Artificial intelligence—augmented clinical decision support systems for pregnancy care: Systematic review. *Journal of Medical Internet Research*, 26, e54737. <https://doi.org/10.2196/54737>
- [10] Rose, S. L., & Shapiro, D. (2024). An ethically supported framework for determining patient notification and informed consent practices when using artificial intelligence in health care. *CHEST*, 166(3), 572–578. <https://doi.org/10.1016/j.chest.2024.04.014>
- [11] Boudierhem, R. (2024). Shaping the future of AI in healthcare through ethics and governance. *Humanities and Social Sciences Communications*, 11(1), 1–12. <https://doi.org/10.1057/s41599-024-02894-w>
- [12] Limaye, D., Limaye, V., Krause, G., & Fortwengel, G. (2017). A systematic review of the

literature to assess self-medication practices. *Annals of Medical and Health Sciences Research*.

<https://www.amhsr.org/articles/a-systematic-review-of-the-literature-to-assess-selfmedication-practices-3711.html>

[13] Ma, C., & Zhang, H. (2025). A cross-sectional study on self-medication with over-the-counter drugs among adolescents: An analysis of current practices. *Frontiers in Public Health*, 13. <https://doi.org/10.3389/fpubh.2025.1560299>

[14] Maever, A., Filipino, C., & Dayrit, M. M. (2023). Self-care practices for common acute conditions in the Philippines: A scoping review. *International Journal of Health Governance*, 28(4). <https://doi.org/10.1108/ijhg-01-2023-0008>

[15] Manila Standard. (2022, January 29). Internet research among reasons why most Filipinos self-medicate. *Manila Standard*. <https://manilastandard.net/?p=314203452>

[16] Decoding OTC medicines in the Philippines. (2023, October 20). NIQ. <https://nielseniq.com/global/en/insights/analysis/2023/decoding-otc-medicines-in-the-philippines/>

[17] Star, S. (2025, January 6). Over 1M Dabawenyos receive free meds from city's Botikang Bayan. *SunStar Publishing Inc*. <https://www.sunstar.com.ph/davao/over-1m-dabawenyos-receive-free-meds-from-citys-botikang-bayan>

[18] Saad, S., & Algarni, A. (2025). Disease prediction from symptom descriptions using deep learning and NLP technique. *International Journal of Advanced Computer Science and Applications*, 16(5). https://thesai.org/Downloads/Volume16No5/Paper_41-Disease_Prediction_from_Symptom_Descriptions.pdf

[19] Hulke, R., & Gudhade, M. (2025). AI-based symptom checker and preliminary diagnosis system. *IOSR Journal of Computer Engineering*, 27(2), 35–42. https://www.iosrjournals.org/manuscript_paper/116690.pdf

[20] Exploring the functionality of AI symptom checkers: How natural language processing transforms patient symptom assessment. (2025, June 7). *Simbo AI Blog*. <https://www.simbo.ai/blog/exploring-the-functionality-of-ai-symptom-checkers-how-natural-language-processing-transforms-patient-symptom-assessment-965754/>

[21] Khaledian, N., & Falahati, S. (2026). Safe and personalized drug recommendation via heterogeneous graph learning: Integrating graphSAGE representations with DDI-aware cross-attention. *Progress in Artificial Intelligence*. <https://doi.org/10.1007/s13748-026-00437-9>

[22] Li, S., Wang, H., He, J., & Chen, X. (2025). AMGNet: An attention-guided multi-graph collaborative decision network for safe medication recommendation. *Electronics*, 14(4), 760. <https://doi.org/10.3390/electronics14040760>

[23] Chen, Q., Li, X., Geng, K., & Wang, M. (2023). Context-aware safe medication recommendations with molecular graph and DDI graph embedding. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(6), 7053–7060. <https://doi.org/10.1609/aaai.v37i6.25861>

- [24] Gallego, F., & Veredas, F. J. (2024). Recognition and normalization of multilingual symptom entities using in-domain-adapted BERT models and classification layers. *Database: The Journal of Biological Databases and Curation*, 2024, baae087. <https://doi.org/10.1093/database/baae087>
- [25] Multilingual health symptom checker. (2025). *Computer Science, Engineering and Technology*, 3(2), 77–81. <https://doi.org/10.46632/cset/3/2/9>
- [26] VerbalCare: A multilingual AI-based healthcare communication system. (2025). *International Journal of Scientific Development and Research*. <https://www.ijedr.org/papers/IJSDR2508203.pdf>
- [27] Kalva, P., & Padakanti, S. (2024). AI-driven personal healthcare: An integrated system for symptom-based OTC medication selection and local availability tracking. *International Journal of Research in Computer Applications and Information Technology*, 7(2), 464–476. https://ijrcait.com/index.php/home/article/view/IJRCAIT_07_02_036
- [28] Sohail, M., et al. (2026). Enhancing medical prescriptions with machine learning: A symptoms-based drug recommendation system. *International Journal of Scientific Research in Engineering and Management*. <https://ijsrem.com/download/enhancing-medical-prescriptions-with-machine-learning-a-symptoms-based-drug-recommendation-system/>
- [29] Bodele, H., Tagde, M., Rangari, S., Jadhao, Y., Bawankar, V., & Kumre, T. (2024). Medicine recommendation system using ML. *International Journal for Research in Applied Science and Engineering Technology*, 12(12), 836–844. <https://doi.org/10.22214/ijraset.2024.65843>
- [30] Zomorodi, M., Ghodsollahee, I., Martin, J. H., Talley, N. J., Salari, V., Plawiak, P., Rahimi, K., & Acharya, U. R. (2023). RECOMED: A comprehensive pharmaceutical recommendation system. *arXiv*. <https://doi.org/10.48550/arXiv.2301.00280>
- [31] Elmes, G. (2025, June 10). OLD CARTS: A framework for clinical excellence. *NursEd*. <https://nursed.ca/virtual-care/oldcarts-patient-assessment-tool/>
- [32] Ummu, M. N. (2023, December 28). OLD CARTS acronym (explained w/ questions). *NurseShip*. <https://www.nurseship.com>
- [33] World Health Organization. (2025, August 21). Dengue. WHO Fact Sheets. <https://www.who.int/news-room/fact-sheets/detail/dengue-and-severe-dengue>
- [34] Centers for Disease Control and Prevention. (2025, August 7). Symptoms of dengue and testing. CDC. <https://www.cdc.gov/dengue/signs-symptoms/index.html>
- [35] Mayo Clinic Staff. (2025, January 18). Diarrhea — Symptoms and causes. Mayo Clinic. <https://www.mayoclinic.org/diseases-conditions/diarrhea/symptoms-causes/syc-20352241>

[36] World Health Organization. (2024, March 7). Diarrhoeal disease. WHO Fact Sheets.

<https://www.who.int/news-room/fact-sheets/detail/diarrhoeal-disease>

[37] Mayo Clinic Staff. (2025, April 12). Sore throat — Symptoms and causes. Mayo Clinic.

<https://www.mayoclinic.org/diseases-conditions/sore-throat/symptoms-causes/syc-20351635>

[38] World Health Organization. (2026, March 24). Tuberculosis. WHO Fact Sheets.

<https://www.who.int/news-room/fact-sheets/detail/tuberculosis>

[39] Centers for Disease Control and Prevention. (2025, January 17). Signs and symptoms of tuberculosis. CDC.

<https://www.cdc.gov/tb/signs-symptoms/index.html>

[40] Mayo Clinic Staff. (2023, August 29). Acute sinusitis — Symptoms and causes. Mayo Clinic.

<https://www.mayoclinic.org/diseases-conditions/acute-sinusitis/symptoms-causes/syc-20351671>

[41] Mayo Clinic Staff. (2019, April 9). Chronic daily headaches — Symptoms and causes. Mayo Clinic.

<https://www.mayoclinic.org/diseases-conditions/chronic-daily-headaches/symptoms-causes/syc-20370891>

[42] World Health Organization. (2025, October 24). Migraine and other headache disorders. WHO Fact Sheets.

<https://www.who.int/news-room/fact-sheets/detail/headache-disorders>

